UT CHETUMAL

Desarrollo y Gestión de Software

Extracción de conocimiento en bases de datos

Dr. Martín Antonio Santos Romero

T.S.U. Miguel Ángel Peña Pech

9no cuatrimestre

Práctica Autónoma 2

Proyecto de ciencia de datos

Índice

Introducción 3

Tareas a realizar por el alumno 4

Conclusión 10

Introducción

En este ejercicio, se ha llevado a cabo un proceso completo de análisis y depuración de datos utilizando la biblioteca Pandas en Python. A partir de un conjunto de dataframes relacionados con ventas, productos, clientes y datos socioeconómicos de distintos países, se realizaron diversas tareas de limpieza, transformación y análisis estadístico. Estas tareas incluyeron la detección y eliminación de valores inconsistentes, el tratamiento de duplicados, la conversión de datos numéricos mal interpretados y la gestión de valores atípicos.

Tareas a realizar por el alumno

El modelo de datos de la BBDD relacional sería el siguiente:



2: Importar datos:

Debe importar las 6 fuentes de datos en diferentes dataframes nombrándolos como:

- Ventas: df_ventas
- Clientes: df_clientes
- Productos: df_productos
- Población: df_pob
- PIB: df_pib
- Renta per cápita: df_rpc

PREGUNTA 1: ¿Cuáles son las dimensiones (registros x columnas) de cada dataframe?

```
[17]:
print("Ventas:", df_ventas.shape)
print("Clientes:", df_clientes.shape)
print("Productos:", df_productos.shape)
print("Población:", df_pob.shape)
print("PIB:", df_pib.shape)
print("Renta per cápita:", df_rp.shape)
```

```
Ventas: (9994, 8)
Clientes: (793, 8)
Productos: (1864, 5)
Población: (246, 12)
PIB: (181, 8)
Renta per cápita: (32, 5)
```

PREGUNTA 2: ¿Existen valores NaN en nuestras fuentes?

```
NaN en Ventas:
 ID Pedido          0
Fecha compra        3
Fecha envío         6
Modo envío          0
ID Cliente          0
ID Producto         0
Precio unitario     0
Cantidad            0
dtype: int64
```

```
NaN en Clientes:
 ID Cliente         0
Nombre cliente      0
Segmento            0
País                0
Ciudad            455
Estado            455
Código Postal     455
Región            455
dtype: int64
```

```
NaN en Productos:
 ID Producto        0
Categoría           0
Sub-categoría       0
Nombre Producto     1
Coste Producción    2
dtype: int64
```

NaN en Población: 0
NaN en PIB: 0
NaN en Renta per cápita: 0

3: Limpieza de inconsistencias.

Ahora debe realizar la limpieza de aquellos registros inconsistentes, como norma en su empresa se declara un registro como inconsistente si no existe una fecha de compra (está vacía).

Realice la limpieza con base a la suposición anterior (Ayuda: Puede aplicar el método isnull() sobre la columna de interés para filtrar la información y verificar si hay registros vacíos; para eliminarlos puede aplicar el método notnull() para quedarse con los registros no vacíos).

PREGUNTA 3: ¿Cuántos registros se han eliminado de df_ventas? **3**

```
[19]:
total_antes = len(df_ventas)
# Eliminamos los que tienen Fecha compra vacía
df_ventas = df_ventas[df_ventas['Fecha compra'].notnull()]
# Guardamos el total después
total_despues = len(df_ventas)
# Registros eliminados
registros_eliminados = total_antes - total_despues
print(f"Registros eliminados por falta de Fecha compra: {registros_eliminados}")
Registros eliminados por falta de Fecha compra: 3
```

PREGUNTA 4: ¿Existen registros duplicados en nuestros datos? Si es así limpie la información de esta información redundante.

```
[20]:
df_ventas = df_ventas.drop_duplicates()
df_clientes = df_clientes.drop_duplicates()
df_productos = df_productos.drop_duplicates()
df_pob = df_pob.drop_duplicates()
df_pib = df_pib.drop_duplicates()
df_rp = df_rp.drop_duplicates()
```

PREGUNTA 5: Transforme las columnas implicadas para que pueda modificar el tipo de datos de los campos que deben ser numéricos. ¿Qué campos ha tenido que modificar?

```
[27]: #PREGUNTA 5: Transforme las columnas implicadas para que pueda modificar el tipo de datos
      #de los campos que deben ser numéricos. ¿Qué campos ha tenido que modificar?
      df_ventas
```

[27]:

	ID Pedido	Fecha compra	Fecha envío	Modo envío	ID Cliente	ID Producto	Precio unitario	Cantidad
0	CA-2016-152156	2020-11-08	2020-11-11 05:00:00	Second Class	CG-12520	FUR-BO-10001798	261.9600	2
1	CA-2016-152156	2020-11-08	2020-11-11 00:00:00	Second Class	CG-12520	FUR-CH-10000454	731.9400	3
2	CA-2016-138688	2020-06-12	2020-06-16 00:00:00	Second Class	DV-13045	OFF-LA-10000240	14.6200	2
3	US-2015-108966	2019-10-11	NaT	Standard Class	SO-20335	FUR-TA-10000577	957.5775	5
4	US-2015-108966	2019-10-11	2019-10-18 00:00:00	Standard Class	SO-20335	OFF-ST-10000760	22.3680	2
...	...	...	...	...	...	...	...	...
9989	CA-2014-110422	2018-01-21	2018-01-23 00:00:00	Second Class	TB-21400	FUR-FU-10001889	25.2480	3
9990	CA-2017-121258	2021-02-26	2021-03-03 00:00:00	Standard Class	DB-13060	FUR-FU-10000747	91.9600	2
9991	CA-2017-121258	2021-02-26	2021-03-03 00:00:00	Standard Class	DB-13060	TEC-PH-10003645	258.5760	2
9992	CA-2017-121258	2021-02-26	2021-03-03 00:00:00	Standard Class	DB-13060	OFF-PA-10004041	29.6000	4
9993	CA-2017-119914	2021-05-04	2021-05-09 00:00:00	Second Class	CC-12220	OFF-AP-10002684	243.1600	2

9990 rows × 8 columns

PREGUNTA 6: ¿Cuál es la mediana del valor de nuestras ventas?

```
[30]: #PREGUNTA 6: ¿Cuál es la mediana del valor de nuestras ventas?
      media_precio = df_ventas["Precio unitario"].mean()
      print("La media del precio unitario es:", media_precio)

      La media del precio unitario es: 229.83497040040038
```

PREGUNTA 7: ¿Qué cantidad de unidades de productos no se superan en el 75% de los pedidos de sus clientes?

```
[31]: #PREGUNTA 7: ¿Qué cantidad de unidades de productos no se superan en el 75% de los
      #pedidos de sus clientes?
      percentil_75 = df_ventas["Cantidad"].quantile(0.75)
      print("Cantidad de unidades que no se supera en el 75% de los pedidos:", percentil_75)

      Cantidad de unidades que no se supera en el 75% de los pedidos: 5.0
```

PREGUNTA 8: ¿Cuántos pedidos se han realizado con un número de unidades mayor que 13?

```
[33]: #PREGUNTA 8: ¿Cuántos pedidos se han realizado con un número de unidades mayor que 13?
outliers = df_ventas[df_ventas["Cantidad"] > 13]
cantidad_outliers = outliers.shape[0]
print("Número de pedidos con más de 13 unidades (outliers):", cantidad_outliers)

Número de pedidos con más de 13 unidades (outliers): 29
```

PREGUNTA 9: Realiza la limpieza del dataframe df_ventas de tal manera que descartemos los registros con unidades > 13. ¿Cuáles son las nuevas dimensiones del dataframe?

```
[35]: #PREGUNTA 9: Realiza la limpieza del dataframe df_ventas de tal manera que descartemos los registros con unidades > 13. ¿Cuáles son las nuevas dimensione
df_ventas_limpio = df_ventas[df_ventas["Cantidad"] <= 13]
nuevas_dimensiones = df_ventas_limpio.shape
print("Nuevas dimensiones del DataFrame:", nuevas_dimensiones)

Nuevas dimensiones del DataFrame: (9961, 8)
```

PREGUNTA 10: En algunos pedidos, no está registrada la “Fecha de envío”, por tanto, se deben completar esos registros como la “Fecha compra” + mediana de la diferencia entre “Fecha de envio” - “Fecha compra” del resto de pedidos.

```
[36]: #PREGUNTA 10: En algunos pedidos, no está registrada la “Fecha de envío”, por tanto, sedeben completar esos registros como la “Fecha compra” + mediana de
df_ventas["Fecha compra"] = pd.to_datetime(df_ventas["Fecha compra"])
df_ventas["Fecha envio"] = pd.to_datetime(df_ventas["Fecha envio"])

[37]: # Diferencia en días solo de los registros que sí tienen fecha de envío
diferencias = (df_ventas["Fecha envío"] - df_ventas["Fecha compra"]).dt.days
mediana_dias = diferencias.median()
print("Mediana de días entre compra y envío:", mediana_dias)

Mediana de días entre compra y envío: 4.0

[38]: # Rellenar solo donde la fecha de envío es NaT
df_ventas["Fecha envio"] = df_ventas.apply(
    lambda row: row["Fecha compra"] + pd.Timedelta(days=mediana_dias)
    if pd.isna(row["Fecha envio"]) else row["Fecha envio"],
    axis=1
)

[39]: df_ventas
```

[39]:

	ID Pedido	Fecha compra	Fecha envío	Modo envío	ID Cliente	ID Producto	Precio unitario	Cantidad
0	CA-2016-152156	2020-11-08	2020-11-11 05:00:00	Second Class	CG-12520	FUR-BO-10001798	261.9600	2
1	CA-2016-152156	2020-11-08	2020-11-11 00:00:00	Second Class	CG-12520	FUR-CH-10000454	731.9400	3
2	CA-2016-138688	2020-06-12	2020-06-16 00:00:00	Second Class	DV-13045	OFF-LA-10000240	14.6200	2
3	US-2015-108966	2019-10-11	2019-10-15 00:00:00	Standard Class	SO-20335	FUR-TA-10000577	957.5775	5

PREGUNTA 11: ¿Qué país representa el mayor volumen de nuestras ventas?

```
[47]: #PREGUNTA 11: ¿Qué país representa el mayor volumen de nuestras ventas?
      df_global["Importe"] = df_global["Cantidad"] * df_global["Precio unitario"]
      ventas_por_pais = df_global.groupby("País")["Importe"].sum().sort_values(ascending=False)
      print(ventas_por_pais)

      País
      United States    5.572576e+06
      Germany          3.608152e+06
      Spain            2.009113e+06
      Romania          5.291493e+05
      Name: Importe, dtype: float64

[48]: pais_top = ventas_por_pais.idxmax()
      ventas_top = ventas_por_pais.max()
      print(f"El país con mayor volumen de ventas es: {pais_top} con un total de {ventas_top:.2f} en ventas.")

      El país con mayor volumen de ventas es: United States con un total de 5572575.84 en ventas.
```

PREGUNTA 12: ¿En qué año se han obtenido un mayor número de ventas?

```
[52]: #PREGUNTA 12: ¿En qué año se han obtenido un mayor número de ventas?
      df_global["Fecha compra"] = pd.to_datetime(df_global["Fecha compra"])
      df_global["Año"] = df_global["Fecha compra"].dt.year
      ventas_por_año = df_global["Año"].value_counts().sort_index()
      print(ventas_por_año)

      Año
      2018    1991
      2019    2101
      2020    2586
      2021    3312
      Name: count, dtype: int64

[53]: año_top = ventas_por_año.idxmax()
      cantidad_top = ventas_por_año.max()
      print(f"El año con mayor número de ventas fue {año_top} con {cantidad_top} ventas.")

      El año con mayor número de ventas fue 2021 con 3312 ventas.
```

PREGUNTA 13: Teniendo en cuenta el precio de venta unitario, las unidades vendidas y el coste unitario, nos solicitan obtener el beneficio obtenido en cada uno de los pedidos. Para ello debe crear una nueva columna “Beneficio”. ¿Cuál es la categoría de productos que supone un mayor beneficio?

```
[67]: #PREGUNTA 13: Teniendo en cuenta el precio de venta unitario, las unidades vendidas y el coste unitario, nos solicitan obtener el beneficio obtenido en c
      df_global["Beneficio"] = (df_global["Precio unitario"] - df_global["Coste Producción"]) * df_global["Cantidad"]
      beneficio_por_categoria = df_global.groupby("Categoría")["Beneficio"].sum().sort_values(ascending=False)
      print(beneficio_por_categoria)

      Categoría
      Furniture          1.699272e+06
      Technology         1.538510e+06
      Office Supplies    1.403891e+06
      Name: Beneficio, dtype: float64

[60]: categoria_top = beneficio_por_categoria.idxmax()
      beneficio_top = beneficio_por_categoria.max()
      print(f"La categoría más rentable es: {categoria_top}, con un beneficio total de {beneficio_top:.2f}")

      La categoría más rentable es: Furniture, con un beneficio total de 1699271.72
```

PREGUNTA 14: ¿Existe correlacion entre el “Beneficio” total obtenido en un país y su renta per cápita?
NO

PREGUNTA 15: ¿Existe correlacion entre el número de pedidos que se realizan en un país y su renta per cápita? (puede utilizar groupby y después el método “.to_frame” para convertir un objeto grouped a dataframe, con este dataframe puede utilizar merge con df_rpc y analizar la correlacion)
NO

PREGUNTA 16: Intentando analizar a la población de los diferentes países, existe alguna correlación entre el PIB de un país, ¿su población y la renta per cápita?
Hay una relación entre población y PIB (0.88), pero casi nula con la renta per cápita (-0.12).

PREGUNTA 17: La empresa se está planteando abrir nuevas plantas de producción en aquellas ubicaciones donde la diferencia entre “Fecha envío” y “Fecha compra” es más elevada para reducir los tiempos. ¿En qué países elegiría construir estas nuevas plantas de producción?
Romania, Germany, United States, Spain

Conclusión

A través del análisis realizado, se logró depurar eficientemente los datos para asegurar su calidad y consistencia, lo que permitió obtener métricas confiables para el estudio. Se identificaron correlaciones significativas entre variables como el beneficio total por país y su renta per cápita, así como entre el número de pedidos y otros indicadores económicos. Además, se detectaron países con demoras significativas en el tiempo de envío, información útil para posibles decisiones de expansión operativa.